FR. CONCEICAO RODRIGUES COLLEGE OF ENGINEERING
Department of Computer Science Engineering

Academic Year 2025-2026	Estimated Time	Experiment No. 2 – 02 Hours	
Course & Semester: T.E. – A and B	Subject Name: Competitive Coding Lab		
Lab 7	Chapter Title: White Box Testing		
Experiment Type: Software Testing & Analysis		Subject Code: 25PECC13CE20	
Name of Student: Aaryan Singh		Roll No. 10281	
Date of Performance: 23/03/2026		Date of Submission: 23/03/2026	
CO Mapping ● CO2: Understand and apply software testing methodologies including white box testing techniques.			

Objective of Experiment:

- To understand the concept and significance of White Box Testing.
- To learn different White Box Testing techniques such as Statement, Branch, and Path Coverage.
- To analyze code paths and identify untested regions in a program.
- To implement and verify White Box Testing on sample programs.

Pre-Requisite:

Basic knowledge of programming (C / C++ / Java / Python), Understanding of control flow, loops, conditionals, and recursion. Familiarity with basic software testing concepts.

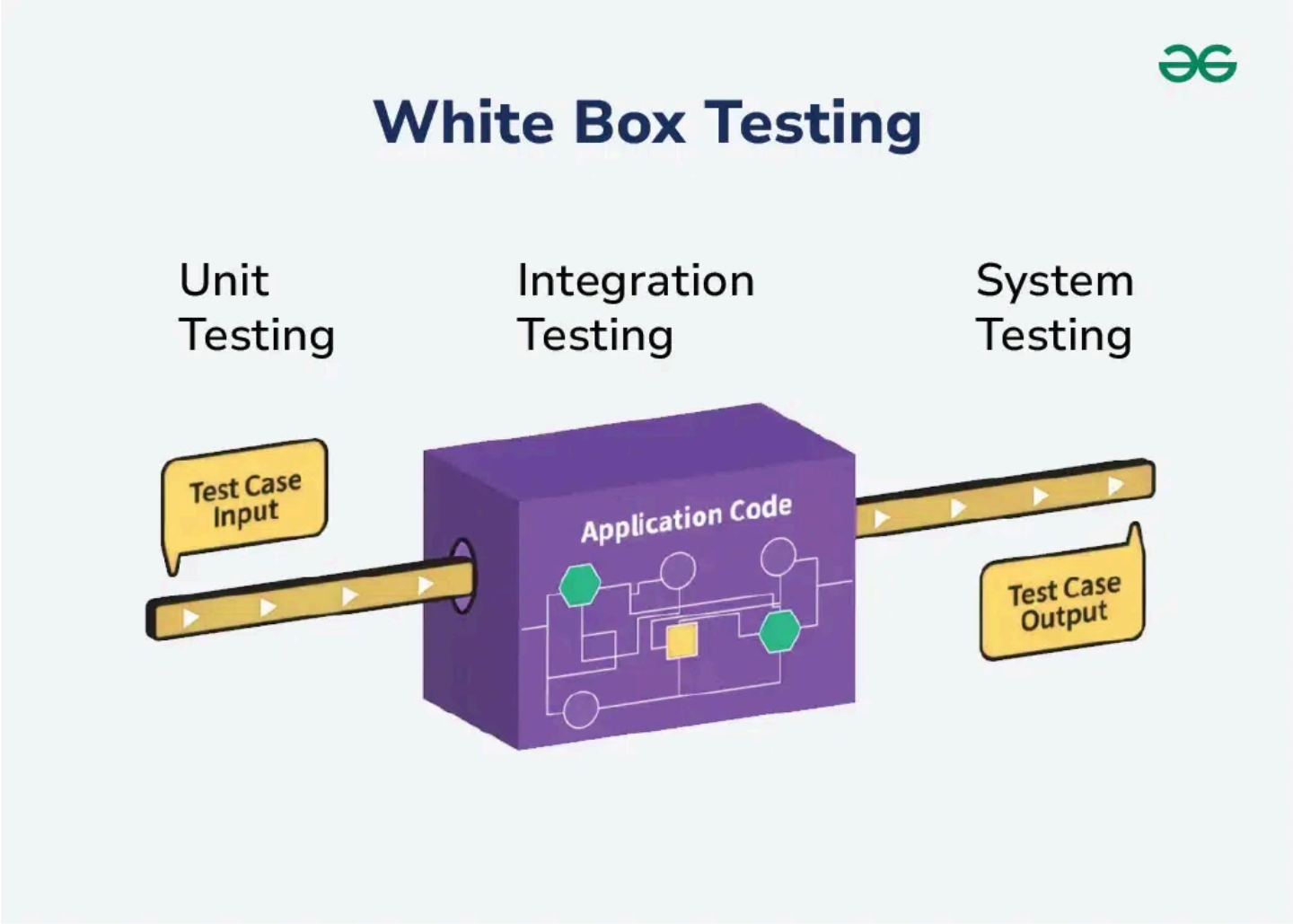
Tools Required:

Compiler / IDE (GCC, CodeBlocks, IntelliJ, VS Code, etc.), JUnit or any unit testing framework.

Theory:

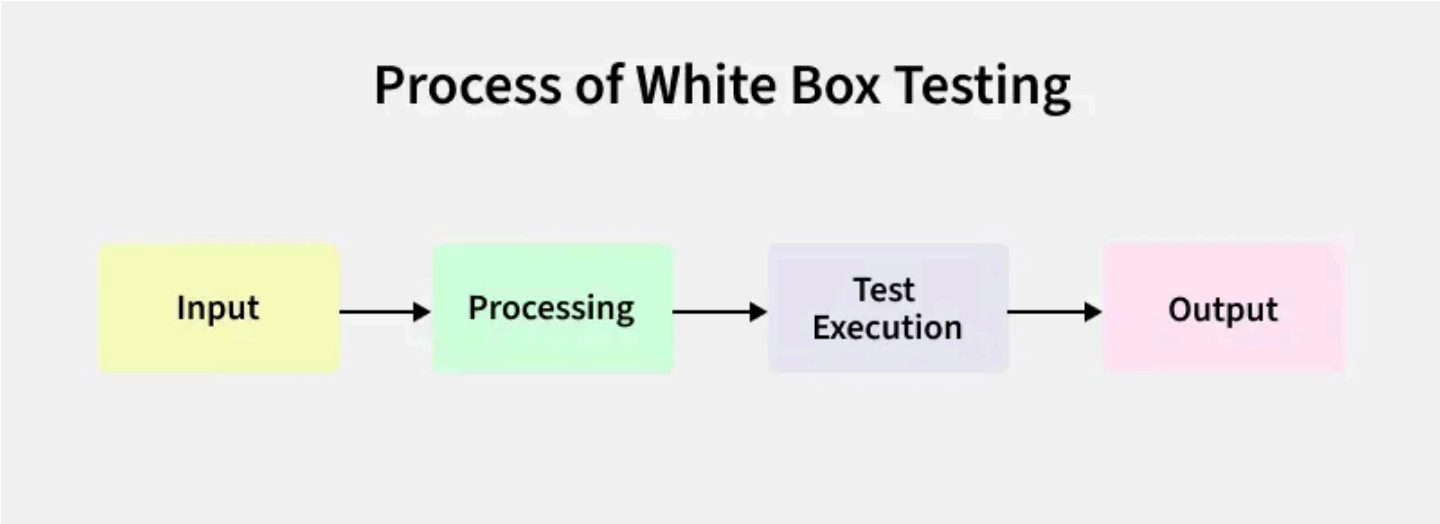
What is White Box Testing?

White Box Testing is a software testing technique that involves testing the internal structure and workings of a software application. The tester has access to the source code and uses this knowledge to design test cases that verify the correctness of the software at the code level. It is also known as Structural Testing or Code-Based Testing.



Why White Box Testing is Important?

- Helps verify internal logic and code paths
- Ensures all code statements and branches are executed
- Detects hidden bugs in logic and flow
- Improves code quality and coverage
- Essential for security and penetration testing



Key Focus Areas of White Box Testing:

Focus Area	Description
Code Logic and Flow	Verifies that control structures like if-else, loops, and switches execute correctly.
Code Coverage	Ensures tests cover statements, branches, and paths in the code.
Data Flow and Variables	Verifies correct initialization, update, and use of variables throughout the program.
Internal Functions	Tests individual methods to confirm they return expected results.
Boundary Conditions	Examines edge cases such as minimum and maximum input values.
Error Handling	Confirms exceptions are caught and invalid inputs are handled gracefully.

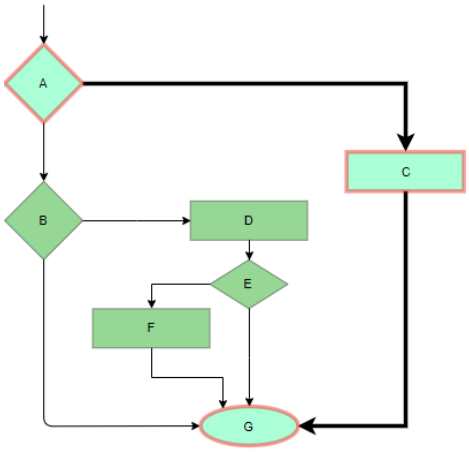
Types of White Box Testing:

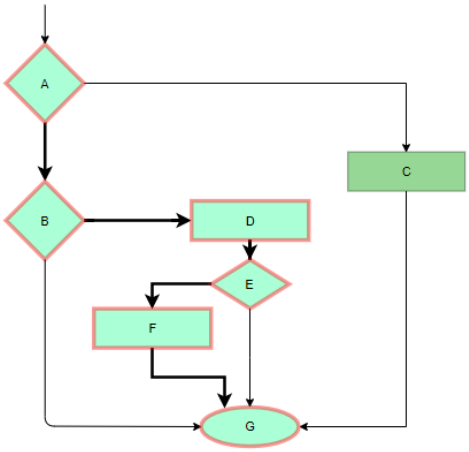
Type	Description
Path Testing	Checks all possible execution paths to ensure every logical condition functions correctly.
Loop Testing	Verifies that loops (for/while) handle variables correctly without infinite loops or logic errors.
Unit Testing	Tests individual functions or components to confirm they work per design requirements.
Mutation Testing	Introduces small code modifications to evaluate the quality of existing test cases.
Integration Testing	Examines how different modules interact and work together after unit testing.
Penetration Testing	Simulates cyber attacks to identify vulnerabilities such as code injection risks and weak APIs.

White Box Testing Techniques:

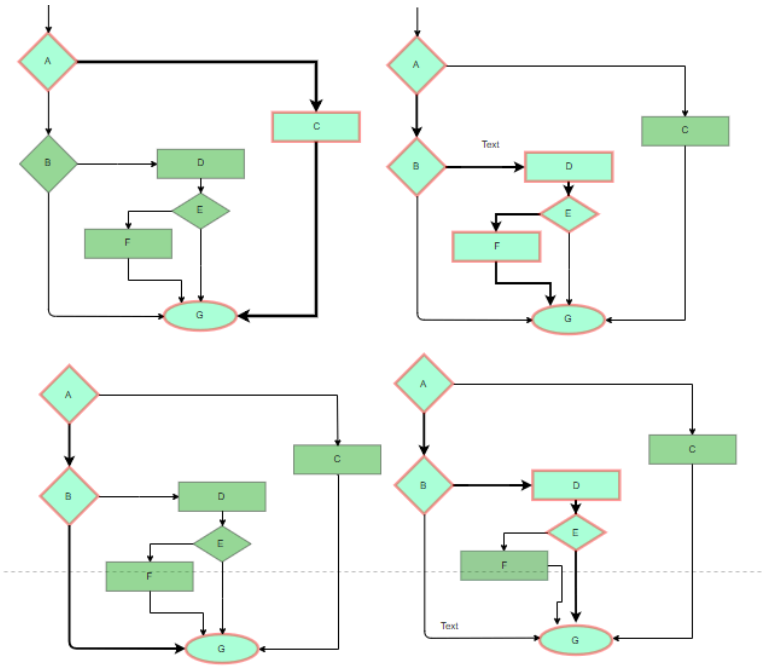
Technique	Goal	Coverage
Statement Coverage	Every line of code is executed at least once	Low – misses branch logic
Branch Coverage	All decision paths (true/false) are tested	Medium – covers if/else branches
Path Coverage	Every possible route through the code is checked	High – catches complex logic bugs
Condition Coverage	Each boolean condition is tested for T and F	High – thorough condition checking

Statement Coverage:





Branch Coverage:



Condition Coverage:
\\Example\\
READ X, Y
IF(X == 0 || Y == 0)
PRINT ‘0’
#TC1 – X = 0, Y = 55
#TC2 – X = 5, Y = 0

Multiple Condition Coverage:
\\Example\\
READ X, Y
IF(X == 0 || Y == 0)
PRINT ‘0’
#TC1: X = 0, Y = 0
#TC2: X = 0, Y = 5
#TC3: X = 55, Y = 0
#TC4: X = 55, Y = 5

Conclusion from Theory:

White Box Testing is essential for verifying the internal logic, control flow, and code paths of a software application. Techniques such as Statement, Branch, and Path Coverage help ensure comprehensive testing, catching hidden bugs that may not surface through Black Box Testing alone.

Conclusion:

Thus, White Box Testing was studied and implemented using Statement Coverage, Branch Coverage, and Path Coverage techniques. It was observed that White Box Testing effectively validates internal logic and control flow, ensuring all code paths are exercised and no untested branches are left hidden. Programs with higher path coverage had fewer undetected logic errors.

On time Completion and Submission (2)	Knowledge of the topic (2)	Implementation and Output (3) Tested with different test cases (3)	Oral (5)	Total (15)

References:

Study Materials	Practice Platforms
● https://www.geeksforgeeks.org ● https://www.w3schools.com ● https://www.softwaretestinghelp.com	● LeetCode ● CodeChef ● HackerRank

Note: Students are expected to paste screenshots of the program with output.